LECTURE 05

React I, *thinking in UI.*

Components, JSX, props, state, and the loop that replaces manual DOM bookkeeping.

components state props JSX Vite

THE PAIN

Vanilla DOM work does not scale kindly.

MANUAL MODE

- × Track state in random variables
- × Find the exact node to update
- × Keep event handlers in sync
- × Patch the page one piece at a time

REACT MODE

- ✓ Store state in the component
- ✓ Describe the UI for that state
- ✓ React calculates the DOM changes
- ✓ Events update state, not random HTML

React is not magic. It is disciplined re-rendering.

DECLARATIVE UI

State describes what the screen should be.

You do not tell React every DOM step. You tell React the result you want for the current data.

event updates state, then the loop runs again.

WHO DOES WHAT?

React is one piece of the workshop.

REACT

Library for building component-based UIs.

JSX

HTML-like syntax inside JavaScript.

BABEL

Compiles JSX into browser-ready JavaScript.

VITE

Runs the dev server and bundles the app.

NODE

Runs JavaScript tooling outside the browser.

NPM

Installs packages and runs project scripts.

JSX IS SYNTAX

It looks like HTML. It compiles to JS.

```jsx
const Clicker = () => {
  return (
    <div className="panel">
      <h1>Clicker</h1>
      <p>Count: {count}</p>
    </div>
  )
}
```

RULES STUDENTS ACTUALLY NEED

- ✓ Return one root element or a fragment.
- ✓ Use {} for JavaScript expressions.
- ✓ Use `className`, not `class`.
- ✓ Close every tag, including inputs.

REUSABLE UI RECIPES

Components are capitalized functions.

```
const App = () => {
  return (
    <>
      <Clicker />
      <Clicker />
      <Clicker />
    </>
  )
}
```

A component lets us package markup, logic, state, and event handlers together.

Lowercase names are treated like HTML tags. Capitalized names are treated like components.

small components are easier to test, read, reuse, and delete.

PARENT TO CHILD

Props configure a component.

```
// Parent
<Clicker name="Mangos" min={0} max={5} />
<Clicker name="Apples" min={1} max={10} />

// Child
const Clicker = ({ name, min, max }) => {
  return <h1>Clicker for {name}</h1>
}
```

PROP RULES

- ✓ Props flow down from parent to child.
- ✓ Props are an object.
- ✓ Props are read-only inside the child.
- ✓ Different props make reusable components useful.

USESTATE

State is data
React watches.

When state changes, React runs the component again and updates the screen to match.

```
const [count, setCount] = useState(0)

const addOne = () => {
  setCount(count + 1)
}
```

do not change state directly. ask React with the setter.

INDEPENDENT INSTANCES

Same component. Different memory.

```
<Clicker name="Mangos" />
<Clicker name="Apples" />
<Clicker name="Peaches" />
```

WHAT STUDENTS SHOULD PICTURE

- ✓ Three component instances render.
- ✓ Each instance has its own count.
- ✓ Clicking one does not update the others.
- ✓ Shared state must move up to a parent.

state belongs to the component that owns the truth.

REACT EVENTS

Pass a function. Do not call it early.

CORRECT

```
<button onClick={addOne}>
  Add one
</button>
```

COMMON BUG

```
<button onClick={addOne()}>
  Add one
</button>
```

The first version says: ”React, run this later when the click happens.” The second version runs during render.

MAP + KEY

Render lists with `array.map()`.

```
const CatList = ({ cats }) => {
  return (
    <ul>
      {cats.map((cat) => (
        <li key={cat.id}>{cat.name}</li>
      ))}
    </ul>
  )
}
```

WHY KEYS MATTER

- ✓ React compares old list to new list.
- ✓ Keys identify each item across renders.
- ✓ Use stable ids when possible.
- ✓ Index keys can cause weird UI bugs when lists reorder.

CHILD TO PARENT SIGNAL

Pass behavior down when state lives up.

```
const App = () => {
  const [cats, setCats] = useState([])
  const addCat = () => setCats([...cats, newCat()])

  return <CatList cats={cats} addCat={addCat} />
}
```

PARENT OWNS

The data and the function that changes it.

CHILD RECEIVES

The current data and a function it can call on click.

REACT I CHECKLIST

The habits that save you later.

DO

- ✓ Name components with capitals.
- ✓ Keep state as small as possible.
- ✓ Update arrays/objects immutably.
- ✓ Use stable keys for lists.
- ✓ Move shared state to the nearest parent.

WATCH FOR

- × Calling event handlers during render.
- × Mutating state with `push` or assignment.
- × Trying to put `if` or `for` statements inside JSX braces.
- × Confusing props with state.